

Gaussowski naiwny klasyfikator Bayesa II

W Gaussowskim klasyfikatorze Naive Bayesa przyjmuje się ciągłe wartości związane z każdą cechą (a posteriori), które mają być rozłożone zgodnie z rozkładem Gaussa..

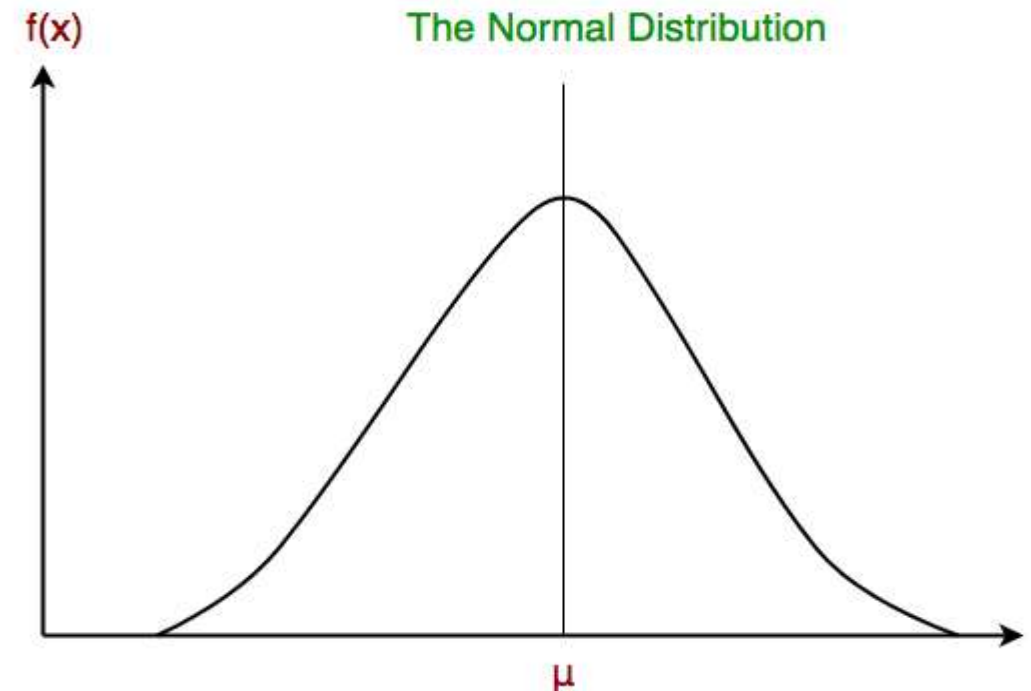
Rozkład Gaussa jest również nazywany **rozkładem normalnym**. Po wykreśleniu daje krzywą w kształcie dzwonu, która jest symetryczna względem średniej wartości cech, jak pokazano poniżej:

Założyliśmy a posteriori, że dystrybucja **zmiennej X1** i **zmiennej X2** w Zalewie Wiślanym jest gaussowska

Twoja własna praca

Script explanation

1. Pythona bibliotek import
2. Dataset import  VLagun_Phys_Years3.csv (change directory to the local)
3. Opis kodu (poniżej - .png files)
4. Wybierz inne **dwie zmienne z bazy danych** do testowania hipotezy o rozkładzie normalnym (Gaussa)
5. Interpretacja wyników: klasyfikacja probalistyczna
6. Powtórz procedurę dla innych dwóch zmiennych z dataset



```
In [1]: # importing required libraries
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
```

```
In [2]: data = pd.read_csv('C:/Users/user/Desktop/Vlagun_Years/Phys/Vlagun_Phys_Years3.csv')
```

```
In [3]: print(data.head())
```

	PSU	O2	temp.	SS	DOC	TPOC	Windspeedinsitu	Depth	Years
0	3.757624	9.46	18.3	52.0	7.500	8.5650	3.5	3.3	0
1	3.504707	9.89	19.1	50.0	7.860	8.5200	0.0	3.6	0
2	3.757624	9.66	18.1	59.0	8.172	8.4915	1.0	3.4	0
3	3.107266	10.36	19.5	46.0	7.848	8.8320	0.0	2.9	0
4	2.619498	11.56	19.0	42.0	7.536	9.2400	0.0	3.0	0

```
In [58]: X = data.iloc[:, :-1]
Y = data.iloc[:, -1]
seed = 7

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
                                                    random_state=seed)
X_train.shape, X_test.shape
```

```
Out[58]: ((84, 8), (36, 8))
```

W klasyfikatorze Naiwnego Bayesa zakłada się, że dane z każdej etykiety są pobierane z prostego rozkładu gaussowskiego. Wyobraźmy sobie, że mamy następujące dane:

```
In [59]: model = GaussianNB()
```

```
In [60]: # fit the model with the training data
model.fit(X_train,y_train)
```

```
Out[60]: GaussianNB(priors=None, var_smoothing=1e-09)
```

```
In [61]: # predict the target on the train dataset
predict_train = model.predict(X_train)
print('Target on train data',predict_train)
```

```
Target on train data [1 0 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 1 0 0 1 1 1 0 0 1 1 0 0 1 0
 1 1 1 0 0 0 1 1 1 0 1 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0 1 0 1 1 0 1 1 0 1 1 1
 0 0 1 1 0 1 1 0 1 0]
```

```
In [62]: # Accuracy Score on train dataset
accuracy_train = accuracy_score(y_train,predict_train)
print('accuracy_score on train dataset : ', accuracy_train)
```

```
accuracy_score on train dataset : 0.9404761904761905
```

```
In [68]: # fit the model with the test data
model.fit(X_test, y_test)
```

```
Out[68]: GaussianNB(priors=None, var_smoothing=1e-09)
```

```
In [69]: # predict the target on the test dataset
predict_test = model.predict(X_test)
print('Target on test data',predict_test)
```

```
Target on test data [1 1 1 1 1 0 1 1 1 0 1 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 1 0 1]
```

```
In [71]: # Accuracy Score on test dataset
accuracy_test = accuracy_score(y_test,predict_test)
print('accuracy_score on test dataset : ', accuracy_test)
```

```
accuracy_score on test dataset : 0.8888888888888888
```

Zmień zmienne
(obiekty), wybierając ich
lokalizację w ramce
danych. Na przykład: **SS** i
TPOC, Depth i PSU itp.

```
In [72]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

```
In [75]: df = pd.read_csv('C:/Users/user/Desktop/Vlagun_Years/Phys/Vlagun_Phys_Years3.csv')
X = df.iloc[:, [6,4]].values      # feature metric
y = df.iloc[:, 8].values          # dependent variable
```

```
In [76]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu');
```



```

n [79]: fig, ax = plt.subplots()

ax.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')
ax.set_title('Naive Bayes Model', size=14)

xlim = (0, 12)
ylim = (6, 11)

xg = np.linspace(xlim[0], xlim[1], 60)
yg = np.linspace(ylim[0], ylim[1], 40)
xx, yy = np.meshgrid(xg, yg)
Xgrid = np.vstack([xx.ravel(), yy.ravel()]).T

for label, color in enumerate(['red', 'blue']):
    mask = (y == label)
    mu, std = X[mask].mean(0), X[mask].std(0)
    P = np.exp(-0.5 * (Xgrid - mu) ** 2 / std ** 2).prod(1)
    Pm = np.ma.masked_array(P, P < 0.03)
    ax.pcolorfast(xg, yg, Pm.reshape(xx.shape), alpha=0.5,
                  cmap=color.title() + 's')
    ax.contour(xx, yy, P.reshape(xx.shape),
               levels=[0.01, 0.1, 0.5, 0.9],
               colors=color, alpha=0.2)

ax.set(xlim=xlim, ylim=ylim)

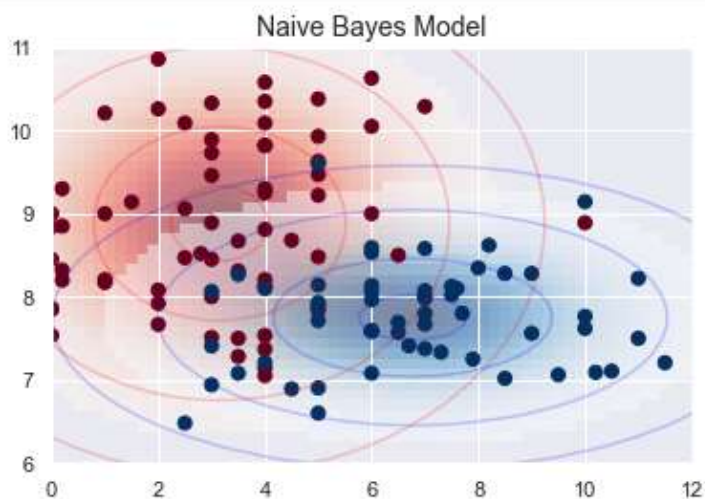
fig.savefig('C:/users/User/Desktop/GNB.png')

```

Jednym z bardzo szybkich sposobów utworzenia prostego modelu jest założenie, że dane są opisywane przez rozkładem gaussowskim bez kowariancji między wymiarami.

Model ten można dopasować, po prostu znajdując **średnią i odchylenie standardowe punktów w każdej etykiecie**, co jest wszystkim, czego potrzeba do zdefiniowania takiego rozkładu.

Wynik tego naiwnego założenia jest przedstawiony na poniższym rysunku:



```
]:
```

```
# The ellipses here represent the Gaussian generative model for each label,
# with larger probability toward the center of the ellipses.
# With this generative model in place for each class,
# we have a simple recipe to compute the likelihood  $P(\text{features} \mid L1)$  for any data point,
# and thus we can quickly compute the posterior ratio and determine which label is the most probable for a given point.

#This procedure is implemented in Scikit-Learn's sklearn.naive_bayes.GaussianNB estimator:
```

```
]:
```

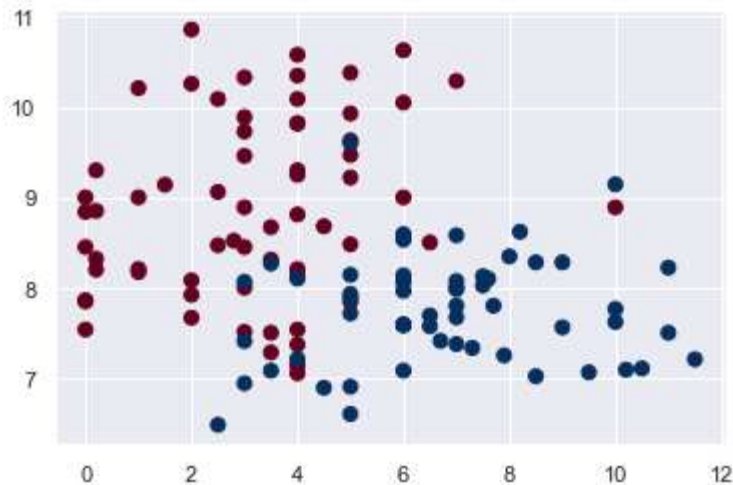
```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X, y);
```

```
]:
```

```
# Now Let's generate some new data and predict the label:
rng = np.random.RandomState(0)
Xnew = [-6, -14] + [14, 18] * rng.rand(2000, 2)
ynew = model.predict(Xnew)
```

```
]:
```

```
# Now we can plot this new data to get an idea of where the decision boundary is:
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='RdBu')
lim = plt.axis()
plt.scatter(Xnew[:, 0], Xnew[:, 1], c=ynew, s=20, cmap='RdBu', alpha=0.1)
plt.axis(lim);
```



```
# We see a slightly curved boundary in the classifications—in general, the boundary in Gaussian naive Bayes is quadratic.
# A nice piece of this Bayesian formalism is that it naturally allows for probabilistic classification,
# which we can compute using the predict_proba method:
yprob = model.predict_proba(Xnew)
yprob[-8:].round(2)
```

Elipsy reprezentują gaussowski model generatywny dla każdej etykiety, z większym prawdopodobieństwem w kierunku środka elipsy.

Mając taki model generatywny dla każdej klasy, mamy prosty przepis na obliczenie prawdopodobieństwa $P(\text{cechy} \mid L1)$ dla dowolnego punktu danych, i w ten sposób możemy szybko obliczyć prawdopodobieństwo a posteriori i określić, która etykieta jest najbardziej prawdopodobna dla danego punktu.

Funkcja **predict_proba()** w klasyfikacji wektorów nośnych (SVC) pakietu Scikit-learn to narzędzie, które pozwala użytkownikom uzyskiwać oszacowania prawdopodobieństwa dla przewidywanych klas.

Rola **predict_proba()**

Funkcja **predict_proba()** została zaprojektowana w celu uzyskania oszacowań prawdopodobieństwa dla każdej etykiety klasy w zadaniu klasyfikacji. Jest to szczególnie przydatne w aplikacjach, w których zrozumienie pewności prognozy jest równie ważne, jak sama prognoza.

Mechanizm wewnętrzny: skalowanie Platt'a

Funkcja **predict_proba()** w SVC wykorzystuje metodę [skalowania Platt'a](#) do konwersji wartości decyzyjnych z SVM na prawdopodobieństwa. Skalowanie Platt'a to etap przetwarzania końcowego, który stosuje model regresji logistycznej do wartości decyzyjnych, czyli odległości próbek od hiperpłaszczyzny.

Po wytrenowaniu, model regresji logistycznej jest używany do konwersji wyników funkcji decyzyjnych na prawdopodobieństwa. Odbywa się to dla każdej klasy, co skutkuje rozkładem prawdopodobieństwa dla wszystkich klas.

Etapy skalowania Platt'a

- Szkolenie SVM** : Początkowo SVM jest szkolony w celu znalezienia optymalnej hiperpłaszczyzny, która oddziela klasy w przestrzeni cech.
- Funkcja decyzyjna**: Funkcja decyzyjna oblicza odległość każdej próbki od hiperpłaszczyzny ze znakiem. Odległość ta jest wykorzystywana jako wartość wejściowa do skalowania Platt'a.
- Regresja logistyczna**: skalowanie Platt'a dopasowuje model regresji logistycznej do wartości decyzyjnych, optymalizując parametry A i B w taki sposób, aby prawdopodobieństwo:

$$P(i = 1 | X) = \frac{1}{1 + \exp(A \cdot f(X) + B)}$$

gdzie $f(X)$ jest wartością decyzyjną dla próbki X

Cechą modelu bayesowskiego jest to, że w naturalny sposób pozwala on na klasyfikację probabilistyczną, którą możemy obliczyć za pomocą metody **predict_proba**:

```
# We see a slightly curved boundary in the classifications—in general, the boundary in Gaussian naive Bayes is quadratic.  
# A nice piece of this Bayesian formalism is that it naturally allows for probabilistic classification,  
# which we can compute using the predict_proba method:  
yprob = model.predict_proba(Xnew)  
yprob[-8:].round(2)
```

```
array([[1., 0.],  
       [1., 0.],  
       [1., 0.],  
       [1., 0.],  
       [1., 0.],  
       [1., 0.],  
       [1., 0.],  
       [1., 0.]])
```

W kolumnach podano prawdopodobieństwa a posteriori odpowiednio pierwszej i drugiej etykiety.

```
# The columns give the posterior probabilities of the first and second label, respectively.  
# If you are looking for estimates of uncertainty in your classification,  
# Bayesian approaches like this can be a useful approach.  
  
# Of course, the final classification will only be as good as the model assumptions that lead to it,  
# which is why Gaussian naive Bayes often does not produce very good results.  
# Still, in many cases—especially as the number of features becomes large—this assumption is not detrimental enough  
# to prevent Gaussian naive Bayes from being a useful method.
```